



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

COMPUTAÇÃO DE ALTO DESEMPENHO

DOCENTE: CARLA DOS SANTOS SANTANA

E SAMUEL XAVIER DE SOUZA

DISCENTE: CLARA VIRGÍNIA NASCIMENTO SANTOS (20240002212)

Tarefa 14

1. ENUNCIADO

Implemente quatro programas MPI com exatamente dois processos. O processo 0 deve enviar uma mensagem ao processo 1, que responde imediatamente com a mesma mensagem. Meça o tempo total de execução de múltiplas trocas consecutivas dessa mensagem, utilizando `MPI_Wtime`. Registre os tempos para diferentes tamanhos, desde mensagens pequenas (como 8 bytes) até mensagens maiores (como 1MB ou mais). Analise graficamente o tempo em função do tamanho da mensagem e identifique os regimes onde a latência domina e onde a largura de banda se torna o fator principal. Explique cada implementação e o que cada função faz. As funções a serem usadas em cada versão são: `MPI_Send`, `MPI_Bsend`, `MPI_Rsend` e `MPI_Ssend`.

2. DISCUSSÃO

Nas tarefas anteriores, foi utilizada a biblioteca OpenMP para paralelizar tarefas dentro de um mesmo programa, em que múltiplas threads compartilhavam a mesma memória. No entanto, o MPI (Message Passing Interface) trabalha de uma forma diferente, sendo um padrão voltado para a memória distribuída. Isto é, ao invés de threads que dividem a mesma memória, o MPI cria múltiplos processos independentes, em que cada um possui sua própria memória isolada, não enxergando os dados dos outros. Dessa maneira, para que esses processos consigam cooperar, é necessário que troquem informações explicitamente através de mensagens, e é justamente isso que essa tarefa investiga. Cada processo recebe um

identificador numérico, chamado de rank. Assim, em uma execução com dois processos, existe o processo de rank 0 e o de rank 1, o que permite definir no código quem envia e quem responde. Nesta tarefa, foi montado o padrão conhecido como ping-pong: o processo 0 envia uma mensagem ao processo 1, que imediatamente a devolve, formando uma ida e volta completa. Esse ciclo é repetido várias vezes a fim de medir um tempo confiável.

2.1. As quatro funções de envio

Embora todas enviem uma mensagem de um processo para outro, cada função de envio do MPI possui um comportamento próprio quanto a quando ela considera o envio concluído e o que ela exige do receptor:

- **MPI_Send** (envio padrão): é o envio mais comum, no qual o próprio MPI decide internamente a melhor estratégia. Geralmente, para mensagens pequenas, ele copia a mensagem em um buffer interno e retorna rapidamente; já para mensagens grandes, tende a esperar o receptor estar pronto antes de retornar.
- **MPI_Bsend** (envio bufferizado): sempre copia a mensagem para um buffer fornecido pelo próprio programador (via **MPI_Buffer_attach**) e retorna imediatamente, independentemente do tamanho. Contudo, essa cópia extra tem um custo, podendo deixar o envio mais lento em mensagens grandes.
- **MPI_Ssend** (envio síncrono): só considera o envio concluído quando o receptor já começou a receber a mensagem. Dessa forma, garante uma sincronização real entre os dois processos, sem usar buffer, mas pode ser mais lento já que o emissor fica esperando o receptor.
- **MPI_Rsend** (envio ready): assume que o recebimento do outro lado já foi postado antes do envio. Caso essa condição não seja respeitada, o comportamento é indefinido (pode gerar erro). Em troca, por dispensar parte da negociação inicial, pode ser ligeiramente mais rápido, todavia é o mais difícil de usar corretamente.

2.2. Latência e Largura de Banda

Para entender os resultados, é preciso considerar dois conceitos fundamentais na comunicação entre processos:

- Latência: é o custo fixo de iniciar uma comunicação, ou seja, o tempo gasto só para "começar a conversa", independentemente do tamanho da mensagem. Esse custo domina quando as mensagens são pequenas.
- Largura de banda (bandwidth): é a velocidade com que os dados conseguem ser transferidos depois que a comunicação já começou, medida em bytes por segundo. Esse fator domina quando as mensagens são grandes.

De maneira análoga, o tempo de uma troca pode ser entendido pela relação: $\text{tempo} = \text{latência} + (\text{tamanho da mensagem} / \text{largura de banda})$. Dessa forma, para mensagens pequenas, o termo da latência é o que pesa, fazendo o tempo ficar praticamente constante. Já para mensagens grandes, o termo da largura de banda cresce proporcionalmente ao tamanho, fazendo o tempo subir de forma linear.

2.3. Funções principais utilizadas

- MPI_Init / MPI_Finalize: ligam e desligam o ambiente MPI; devem ser sempre a primeira e a última coisa do programa.
- MPI_Comm_rank: informa o identificador (rank) do processo atual, permitindo separar o que o processo 0 e o processo 1 fazem.
- MPI_Send / MPI_Recv: enviam e recebem a mensagem, respectivamente.
- MPI_Wtime: funciona como um cronômetro de alta precisão do próprio MPI, retornando o tempo em segundos.
- MPI_Barrier: faz todos os processos esperarem uns aos outros em um ponto, servindo para sincronizar antes de iniciar a medição.
- MPI_Buffer_attach / MPI_Buffer_detach: entregam e devolvem o buffer que o MPI_Bsend precisa para funcionar.
- MPI_Irecv / MPI_Wait: usados na versão do MPI_Rsend, em que o recebimento é postado antecipadamente (MPI_Irecv) e depois aguardado (MPI_Wait), garantindo que o receptor já esteja pronto.

2.4. ETAPAS PARA RESOLUÇÃO DA TAREFA

- 1º) Escrever o programa base do ping-pong com dois processos (envio e resposta);
- 2º) Variar o tamanho da mensagem (de 8 bytes até 1MB);
- 3º) Repetir cada troca várias vezes e medir o tempo total com MPI_Wtime;
- 4º) Criar as quatro versões, trocando apenas a função de envio (Send, Bsend, Rsend, Ssend);
- 5º) Coletar os tempos de cada versão e tamanho;
- 6º) Montar o gráfico do tempo em função do tamanho da mensagem;
- 7º) Identificar os regimes de latência e de largura de banda;
- 8º) Escrever a análise e a conclusão.

2.5. COMPILAR

```
gcc tarefa14_send.c -o tarefa14_send.exe -lmsmpi  
mpiexec -n 2 tarefa14_send.exe
```

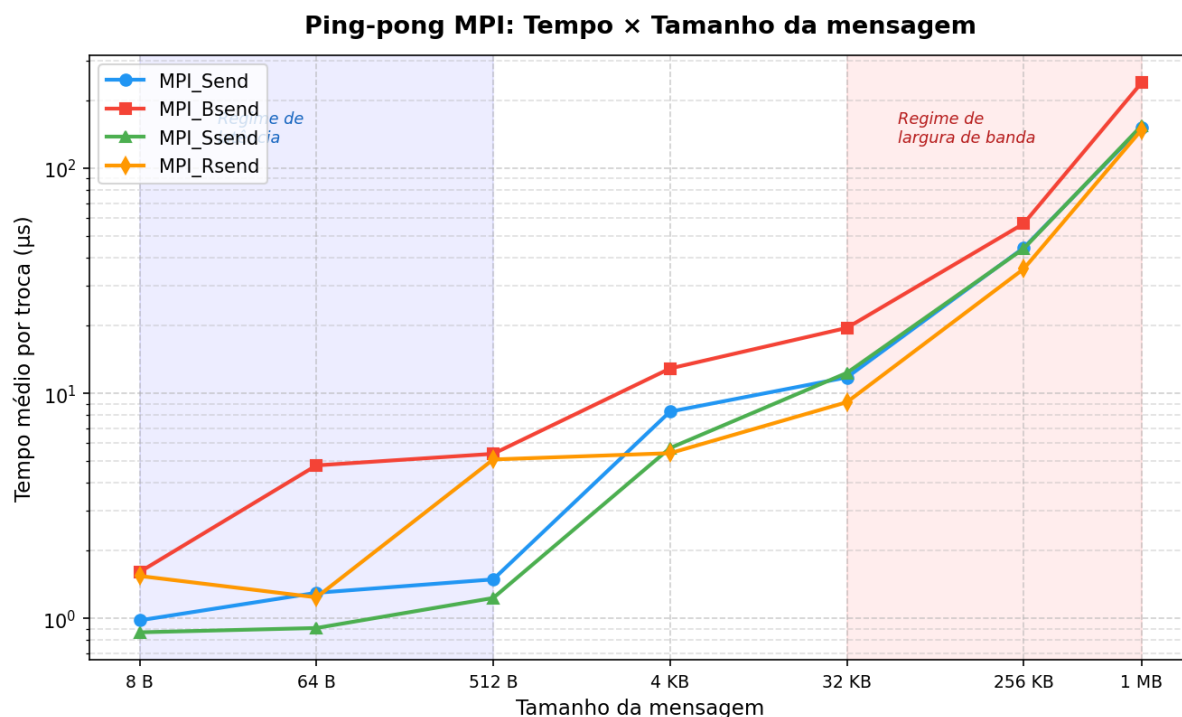
2.6. RESOLUÇÃO EM CÓDIGO DA TAREFA

Segue no apêndice.

2.7. OUTPUT

Segue no apêndice.

2.8. GRÁFICO



3. CONCLUSÃO

Os quatro programas demonstram de forma clara como o tamanho da mensagem define qual fator limita o desempenho da comunicação MPI. Para mensagens pequenas, de 8 bytes até 512 bytes, o gargalo é a latência, com tempos estáveis em torno de 1 a 5 μs independentemente do tamanho. Já a partir de 4 KB, o gargalo passa a ser a largura de banda, com os tempos crescendo proporcionalmente ao tamanho da mensagem e chegando a valores de até 239 μs em 1 MB no caso do MPI_Bsend.

Além disso, a comparação entre as quatro funções de envio evidenciou que a escolha da função impacta diretamente o desempenho. O MPI_Bsend pagou o preço mais alto pela sua cópia extra em memória, especialmente em mensagens grandes. O MPI_Ssend, embora exija sincronização com o receptor, se mostrou competitivo por rodar em memória compartilhada, onde esse custo é baixo. O MPI_Rsend foi o mais rápido em 1 MB, contudo o mais difícil de usar com segurança, já que exigiu a implementação de MPI_Irecv e MPI_Barrier para garantir que o recebimento estivesse pronto antes do envio, o que adicionou variação nos resultados menores. O MPI_Send, por sua vez, apresentou o comportamento mais estável e previsível de todos. Portanto, conclui-se que não basta apenas escolher a função de envio mais rápida em teoria, sendo essencial entender o regime de operação do programa e os custos reais de cada estratégia de comunicação.

4. APÊNDICE

4.1. RESOLUÇÃO COM MPI_SEND

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h> // biblioteca do MPI

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv); // liga o MPI, sempre tem que vir primeiro

    int rank; // pra saber quem eu sou: processo 0 ou processo 1
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // tamanhos de mensagem em bytes, do pequeninho ate 1MB
    int tamanhos[] = {8, 64, 512, 4096, 32768, 262144, 1048576};
    int qtd = 7;
    int repeticoes = 1000; // quantas idas e voltas pra cada tamanho (pra
    dar pra medir direito)

    if (rank == 0) {
        printf("Versao MPI_Send\n");
    }

    // loop que passa por cada tamanho de mensagem
    for (int t = 0; t < qtd; t++) {
        int tam = tamanhos[t];

        // cria a mensagem com tam bytes
        char *mensagem = (char*) malloc(tam);
        for (int i = 0; i < tam; i++) {
            mensagem[i] = 'a'; // enche de qualquer coisa, o conteudo nao
            importa
        }

        MPI_Barrier(MPI_COMM_WORLD); // espera os 2 processos chegarem aqui
        antes de medir
        double inicio = MPI_Wtime(); // cronometro do MPI ligando

        // aqui acontece o ping-pong varias vezes seguidas
        for (int r = 0; r < repeticoes; r++) {
            if (rank == 0) {
                // processo 0 manda e depois espera a resposta voltar
                MPI_Send(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                MPI_Recv(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD,
                MPI_STATUS_IGNORE);
            } else {
                // processo 1 espera chegar e ja devolve a mesma mensagem
            }
        }
    }
}
```

```

        MPI_Recv(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Send(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }
}

double fim = MPI_Wtime(); // cronometro desligando
double tempo_total = fim - inicio;

// so o processo 0 imprime, pra nao sair tudo duplicado na tela
if (rank == 0) {
    // tempo de 1 ida e volta, em microseg
    double tempo_medio_us = (tempo_total / repeticoes) * 1000000.0;
    // largura de banda em MB/s -> vai e volta = 2x o tamanho
    double banda = (2.0 * tam * repeticoes) / tempo_total / (1024 *
1024);

    printf("Tamanho: %8d bytes | Tempo medio por troca: %10.3f us |
Banda: %8.2f MB/s\n",
        tam, tempo_medio_us, banda);
}

free(mensagem);
}

MPI_Finalize(); // desliga o MPI, sempre por ultimo
return 0;
}

```

4.2. RESOLUÇÃO COM MPI_Bsend

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int tamanhos[] = {8, 64, 512, 4096, 32768, 262144, 1048576};
    int qtd = 7;
    int repeticoes = 1000;

    // o Bsend precisa de um buffer NOSSO pra ele copiar a mensagem antes
    de mandar.

```

```

    // esse buffer tem que caber a maior mensagem (1MB) + uma folga que o
MPI exige (MPI_BSEND_OVERHEAD)
    int tam_buffer = 1048576 + MPI_BSEND_OVERHEAD;
    char *buffer = (char*) malloc(tam_buffer);
    MPI_Buffer_attach(buffer, tam_buffer); // entrega o buffer pro MPI usar
nos Bsend

    if (rank == 0) {
        printf("Versao MPI_Bsend\n");
    }

    for (int t = 0; t < qtd; t++) {
        int tam = tamanhos[t];
        char *mensagem = (char*) malloc(tam);
        for (int i = 0; i < tam; i++) {
            mensagem[i] = 'a';
        }

        MPI_Barrier(MPI_COMM_WORLD);
        double inicio = MPI_Wtime();

        for (int r = 0; r < repeticoes; r++) {
            if (rank == 0) {
                MPI_Bsend(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
// manda usando o buffer
                MPI_Recv(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            } else {
                MPI_Recv(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
                MPI_Bsend(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
            }
        }

        double fim = MPI_Wtime();
        double tempo_total = fim - inicio;

        if (rank == 0) {
            double tempo_medio_us = (tempo_total / repeticoes) * 1000000.0;
            double banda = (2.0 * tam * repeticoes) / tempo_total / (1024 *
1024);
            printf("Tamanho: %8d bytes | Tempo medio por troca: %10.3f us |
Banda: %8.2f MB/s\n", tam, tempo_medio_us, banda);
        }

        free(mensagem);
    }

```



```

    MPI_Buffer_detach(&buffer, &tam_buffer); // devolve o buffer pro MPI
antes de fechar
    free(buffer);

    MPI_Finalize();
    return 0;
}

```

4.3. RESOLUÇÃO COM MPI_Ssend

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int tamanhos[] = {8, 64, 512, 4096, 32768, 262144, 1048576};
    int qtd = 7;
    int repeticoes = 1000;

    if (rank == 0) {
        printf("Versao MPI_Ssend\n");
    }

    for (int t = 0; t < qtd; t++) {
        int tam = tamanhos[t];
        char *mensagem = (char*) malloc(tam);
        for (int i = 0; i < tam; i++) {
            mensagem[i] = 'a';
        }

        MPI_Barrier(MPI_COMM_WORLD);
        double inicio = MPI_Wtime();

        for (int r = 0; r < repeticoes; r++) {
            if (rank == 0) {
                // o Ssend so termina quando o processo 1 JA comecou a
receber (sincrono)
                MPI_Ssend(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                MPI_Recv(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
            } else {

```

```

        MPI_Recv(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        MPI_Ssend(mensagem, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }
}

double fim = MPI_Wtime();
double tempo_total = fim - inicio;

if (rank == 0) {
    double tempo_medio_us = (tempo_total / repeticoes) * 1000000.0;
    double banda = (2.0 * tam * repeticoes) / tempo_total / (1024 *
1024);
    printf("Tamanho: %8d bytes | Tempo medio por troca: %10.3f us |
Banda: %8.2f MB/s\n", tam, tempo_medio_us, banda);
}

free(mensagem);
}

MPI_Finalize();
return 0;
}

```

4.4. RESOLUÇÃO COM MPI_Rsend

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int tamanhos[] = {8, 64, 512, 4096, 32768, 262144, 1048576};
    int qtd = 7;
    int repeticoes = 1000;

    if (rank == 0) {
        printf("Versao MPI_Rsend\n");
    }

    for (int t = 0; t < qtd; t++) {
        int tam = tamanhos[t];
        // aqui preciso de 2 buffers: um pra mandar e outro pra receber
    }
}

```

```

        // isso porque o recebimento fica "aberto" (Irecv) enquanto eu
mando,
        // entao nao da pra usar o mesmo buffer pras duas coisas ao mesmo
tempo.

        char *mensagem = (char*) malloc(tam);
        char *recebido = (char*) malloc(tam);
        for (int i = 0; i < tam; i++) {
            mensagem[i] = 'a';
        }

        MPI_Barrier(MPI_COMM_WORLD);
        double inicio = MPI_Wtime();

        for (int r = 0; r < repeticoes; r++) {
            // o Rsend SO funciona se o recebimento do outro lado JA
estiver postado.
            // por isso posta o recv antes (MPI_Irecv) e usa o barrier pra
garantir
            // que os 2 lados ja postaram o recv, ai sim pode mandar com
seguranca.

            MPI_Request pedido;
            if (rank == 0) {
                MPI_Irecv(recebido, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD,
&pedido); // ja deixa o recv pronto
                MPI_Barrier(MPI_COMM_WORLD); // garante que o processo 1
tambem ja postou o dele
                MPI_Rsend(mensagem, tam, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
// agora pode mandar
                MPI_Wait(&pedido, MPI_STATUS_IGNORE); // espera a resposta
chegar
            } else {
                MPI_Irecv(recebido, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
&pedido);

                MPI_Barrier(MPI_COMM_WORLD);
                MPI_Wait(&pedido, MPI_STATUS_IGNORE); // espera a mensagem
do processo 0 chegar
                MPI_Rsend(recebido, tam, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
// devolve a mesma mensagem
            }
        }

        double fim = MPI_Wtime();
        double tempo_total = fim - inicio;

        if (rank == 0) {
            double tempo_medio_us = (tempo_total / repeticoes) * 1000000.0;
            double banda = (2.0 * tam * repeticoes) / tempo_total / (1024 *
1024);

```

```

        printf("Tamanho: %8d bytes | Tempo medio por troca: %10.3f us |
Banda: %8.2f MB/s\n", tam, tempo_medio_us, banda);
    }

    free(mensagem);
    free(recebido);
}

MPI_Finalize();
return 0;
}

```

4.5. OUTPUT COM MPI_Send

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> gcc tarefa14_mpisend.c -o tarefa14_mpisend.exe -lmmpi
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> mpiexec -n 2 tarefa14_mpisend.exe
Versao MPI_Send
Tamanho:      8 bytes | Tempo medio por troca:      2.066 us | Banda:      7.39 MB/s
Tamanho:     64 bytes | Tempo medio por troca:      2.158 us | Banda:     56.58 MB/s
Tamanho:    512 bytes | Tempo medio por troca:      0.765 us | Banda:   1276.72 MB/s
Tamanho:   4096 bytes | Tempo medio por troca:     11.928 us | Banda:    654.95 MB/s
Tamanho:  32768 bytes | Tempo medio por troca:     24.371 us | Banda:   2564.53 MB/s
Tamanho: 262144 bytes | Tempo medio por troca:     84.331 us | Banda:  5929.04 MB/s
Tamanho:1048576 bytes | Tempo medio por troca:    157.577 us | Banda: 12692.18 MB/s
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14>

```

4.6. OUTPUT COM MPI_Bsend

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> gcc tarefa14_mpibsend.c -o tarefa14_mpibsend.exe -lmmpi
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> mpiexec -n 2 tarefa14_mpibsend.exe
Versao MPI_Bsend
Tamanho:      8 bytes | Tempo medio por troca:      1.607 us | Banda:      9.50 MB/s
Tamanho:     64 bytes | Tempo medio por troca:      4.783 us | Banda:     25.52 MB/s
Tamanho:    512 bytes | Tempo medio por troca:      5.385 us | Banda:    181.36 MB/s
Tamanho:   4096 bytes | Tempo medio por troca:     12.892 us | Banda:    605.98 MB/s
Tamanho:  32768 bytes | Tempo medio por troca:     19.528 us | Banda:   3200.55 MB/s
Tamanho: 262144 bytes | Tempo medio por troca:     56.728 us | Banda:  8814.02 MB/s
Tamanho:1048576 bytes | Tempo medio por troca:    239.734 us | Banda: 8342.57 MB/s
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14>

```

4.7. OUTPUT COM MPI_Ssend

```

PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> mpiexec -n 2 tarefa14_mpissend.exe
Versao MPI_Ssend
Tamanho:      8 bytes | Tempo medio por troca:      0.867 us | Banda:     17.60 MB/s
Tamanho:     64 bytes | Tempo medio por troca:      0.906 us | Banda:    134.78 MB/s
Tamanho:    512 bytes | Tempo medio por troca:      1.232 us | Banda:    792.60 MB/s
Tamanho:   4096 bytes | Tempo medio por troca:      5.712 us | Banda:   1367.64 MB/s
Tamanho:  32768 bytes | Tempo medio por troca:     12.320 us | Banda:   5073.09 MB/s
Tamanho: 262144 bytes | Tempo medio por troca:     43.921 us | Banda:  11384.02 MB/s
Tamanho:1048576 bytes | Tempo medio por troca:    154.511 us | Banda: 12944.04 MB/s
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14>

```

4.8. OUTPUT COM MPI_Rsend

```
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> gcc tarefa14_mpi.c -o tarefa14_mpi.exe -lmsmpi
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> mpiexec -n 2 tarefa14_mpi.exe
Versao MPI_Rsend
Tamanho:      8 bytes | Tempo medio por troca:      3.403 us | Banda:      4.48 MB/s
Tamanho:     64 bytes | Tempo medio por troca:      0.882 us | Banda:    138.48 MB/s
Tamanho:    512 bytes | Tempo medio por troca:      1.708 us | Banda:    571.82 MB/s
Tamanho:   4096 bytes | Tempo medio por troca:     12.609 us | Banda:    619.60 MB/s
Tamanho:  32768 bytes | Tempo medio por troca:     24.948 us | Banda:   2505.19 MB/s
Tamanho: 262144 bytes | Tempo medio por troca:     54.897 us | Banda:   9107.97 MB/s
Tamanho:1048576 bytes | Tempo medio por troca:    188.639 us | Banda: 10602.24 MB/s
PS Z:\Documents\UFRN\TI\Período 5\CAD\tarefa14> |
```